

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO10 – Develop Program Using Loop Structure

Nominal Duration: Theory 3.5 Hours / Practical 6 Hours

L.O 10: DEVELOP PROGRAM USING LOOP STRUCTURE

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Explain the purpose of loop structures in Python.
2. Initialize variables for loop operations.
3. Develop programs using while loops.
4. Apply conditions to control while loop execution.
5. Increment or decrement counter variables correctly.
6. Display statements repeatedly while conditions are true.
7. Apply the break statement.
8. Apply the continue statement.
9. Apply while-else structures.
10. Develop programs using for loops.
11. Apply the range() function with for loops.
12. Apply for loops to lists and other collections.
13. Combine loops with conditional statements.
14. Develop nested loop structures.
15. Test and debug loop programs.
16. Identify and correct infinite-loop and counter errors.
17. Apply loops to repetitive programming tasks.

INFORMATION SHEET No. 10.1

Title: Understanding Python Loop Structures

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain the concept and purpose of loops.
- Explain the while loop structure.
- Initialize and use loop counters.
- Apply loop conditions.
- Increment and decrement variables.
- Use break.
- Use continue.
- Use else with a while loop.
- Explain common while loop errors.
- Explain the for loop structure.
- Apply range().
- Use for loops with collections.
- Combine loops with conditions.
- Explain nested loops.
- Apply loops to repetitive programming tasks.

1. Introduction to Loop Structures

A loop is a programming structure used to repeat a block of instructions.

Without a loop, repeated instructions may need to be written many times.

Example without a loop:

```
print("Python")
print("Python")
print("Python")
print("Python")
print("Python")
```

A loop can perform the same task more efficiently.

Example:

```
for i in range(5):
    print("Python")
```

Output:

```
Python
Python
Python
Python
Python
```

2. Why Loops Are Used

Loops are useful when a program needs to:

- Repeat an operation.
- Count values.
- Process many items.
- Display repeated information.
- Read elements from a collection.
- Search data.
- Perform calculations repeatedly.
- Generate patterns.
- Process records.
- Control repetitive tasks.

Python provides two main loop structures:

1. `while`
2. `for`

3. The while Loop

A while loop repeatedly executes a block of code while a condition remains True.

Syntax:

```
while condition:  
    statements
```

Example:

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

Output:

```
1  
2  
3  
4  
5
```

4. Components of a while Loop

A basic counter-controlled while loop usually contains:

1. Initialization
2. Condition
3. Loop body
4. Increment or decrement

Example:

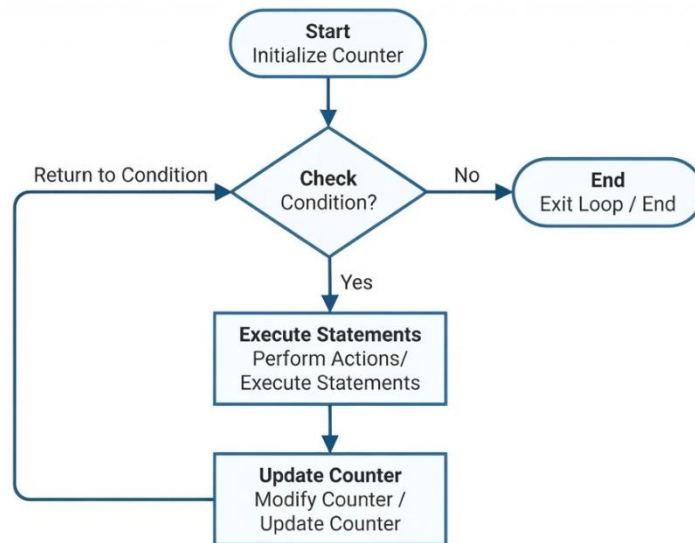
```

count = 1                # Initialization

while count <= 5:        # Condition
    print(count)         # Loop body
    count += 1           # Increment

```

The general execution process is:



5. Initializing a Loop Variable

Before using a counter in a while loop, it is normally initialized.

Example:

```
count = 1
```

This gives count an initial value.

Other examples:

```

number = 0
attempt = 1
index = 0

```

6. Checking a while Condition

The condition determines whether the loop continues.

Example:

```

count = 1

while count <= 5:
    print(count)
    count += 1

```

Python repeatedly checks:

```
count <= 5
```

When the condition becomes False, the loop stops.

7. Incrementing a Counter

An increment increases a variable value.

Example:

```
count += 1
```

This is equivalent to:

```
count = count + 1
```

Example:

```
count = 1
```

```
while count <= 5:  
    print("Count:", count)  
    count += 1
```

8. Decrementing a Counter

A counter can also decrease.

Example:

```
count = 5
```

```
while count >= 1:  
    print(count)  
    count -= 1
```

Output:

```
5  
4  
3  
2  
1
```

9. Printing While the Condition Is True

Example:

```
count = 1
```

```
while count <= 3:  
    print("Condition is true")  
    count += 1
```

Output:

```
Condition is true  
Condition is true  
Condition is true
```

The statement is executed each time the condition is true.

10. How a while Loop Stops

Consider:

```
count = 1

while count <= 3:
    print(count)
    count += 1
```

Execution:

Iteration	count	count <= 3	Action
1	1	True	Print 1
2	2	True	Print 2
3	3	True	Print 3
4	4	False	Stop

The counter must be updated correctly so that the condition eventually becomes false.

11. Infinite Loops

An infinite loop continues indefinitely because its stopping condition never becomes false.

Incorrect:

```
count = 1

while count <= 5:
    print(count)
```

The value of count never changes.

Therefore:

```
count <= 5
```

remains true.

Correct:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

12. The break Statement

The break statement immediately stops the nearest enclosing loop.

Example:

```
count = 1

while count <= 10:
    print(count)

    if count == 5:
        break

    count += 1
```

Output:

```
1
2
3
4
5
```

When count reaches 5, the loop stops.

13. break with a Condition

Example:

```
number = 1

while number <= 10:
    if number == 6:
        break

    print(number)
    number += 1
```

Output:

```
1
2
3
4
5
```

14. The continue Statement

The continue statement skips the remaining statements in the current iteration and proceeds to the next iteration.

Example:

```
count = 0

while count < 5:
    count += 1

    if count == 3:
        continue

    print(count)
```

Output:

```
1
2
4
5
```

The value 3 is skipped.

15. Important continue Rule in while Loops

When using continue with a counter-controlled while loop, the counter should normally be updated before continue can repeatedly bypass the update.

Incorrect:

```
count = 1

while count <= 5:
    if count == 3:
        continue

    print(count)
    count += 1
```

When count becomes 3, continue runs before the counter is incremented.

This can create an infinite loop.

Correct:

```
count = 0

while count < 5:
    count += 1

    if count == 3:
        continue

    print(count)
```

16. while Loop with else

Python allows an else block after a while loop.

Example:

```
count = 1

while count <= 3:
    print(count)
    count += 1
else:
    print("Loop completed")
```

Output:

```
1
2
3
Loop completed
```

The else block runs when the while condition becomes false normally.

17. while-else with break

If the loop is terminated using break, the else block is not executed.

Example:

```

count = 1

while count <= 5:
    if count == 3:
        break

    print(count)
    count += 1
else:
    print("Loop completed")

```

Output:

```

1
2

```

The else block is skipped because break ended the loop.

18. Conditional Expressions in while Loops

Comparison operators are commonly used in loop conditions.

Examples:

```

<
>
<=
>=
==
!=

```

Example:

```

count = 1

while count < 10:
    print(count)
    count += 2

```

Output:

```

1
3
5
7
9

```

19. Logical Operators in Loop Conditions

Logical operators can combine multiple conditions.

Example:

```

count = 1

while count <= 10 and count != 6:
    print(count)
    count += 1

```

The loop continues only while both conditions remain true.

20. Practical while Loop — Sum Numbers

Example:

```
count = 1
total = 0

while count <= 5:
    total += count
    count += 1

print("Total:", total)
```

Output:

Total: 15

21. Practical while Loop — Password Attempts

Example:

```
attempt = 1

while attempt <= 3:
    print("Login attempt:", attempt)
    attempt += 1
```

This demonstrates a fixed number of repeated attempts.

22. Common while Loop Errors

Common errors include:

- Forgetting initialization.
- Incorrect condition.
- Forgetting increment/decrement.
- Updating the wrong variable.
- Incorrect indentation.
- Creating an infinite loop.
- Using continue before counter update.
- Incorrect comparison operator.
- Using = instead of ==.
- Misusing break.

23. Introduction to the for Loop

A for loop is used to iterate over a sequence or other iterable.

Syntax:

```
for variable in iterable:
    statements
```

Example:

```
for number in [1, 2, 3]:
    print(number)
```

Output:

```
1
2
3
```

24. for Loop with a List

Example:

```
students = ["Dara", "Sokha", "Vanna"]

for student in students:
    print(student)
```

Output:

```
Dara
Sokha
Vanna
```

Each item is assigned to:

```
student
```

one at a time.

25. for Loop with a String

A string can also be iterated character by character.

Example:

```
for character in "Python":
    print(character)
```

Output:

```
P
y
t
h
o
n
```

26. The range() Function

range() generates a sequence of integer values commonly used with for loops.

Basic syntax:

```
range(stop)
```

Example:

```
for number in range(5):
    print(number)
```

Output:

```
0
1
```

2
3
4

The ending value is excluded.

27. range(start, stop)

Syntax:

```
range(start, stop)
```

Example:

```
for number in range(1, 6):  
    print(number)
```

Output:

1
2
3
4
5

28. range(start, stop, step)

Syntax:

```
range(start, stop, step)
```

Example:

```
for number in range(2, 11, 2):  
    print(number)
```

Output:

2
4
6
8
10

29. Counting Backward with range()

A negative step can count backward.

Example:

```
for number in range(5, 0, -1):  
    print(number)
```

Output:

5
4
3
2
1

30. for Loop with Conditional Statements

A conditional statement can be placed inside a for loop.

Example:

```
for number in range(1, 6):  
    if number % 2 == 0:  
        print(number, "is even")  
    else:  
        print(number, "is odd")
```

Output:

```
1 is odd  
2 is even  
3 is odd  
4 is even  
5 is odd
```

31. break with a for Loop

Example:

```
for number in range(1, 10):  
    if number == 5:  
        break  
  
    print(number)
```

Output:

```
1  
2  
3  
4
```

32. continue with a for Loop

Example:

```
for number in range(1, 6):  
    if number == 3:  
        continue  
  
    print(number)
```

Output:

```
1  
2  
4  
5
```

33. Looping Through List Data

Example:

```
scores = [80, 75, 90, 85]
```

```
for score in scores:
    print("Score:", score)
```

Output:

```
Score: 80
Score: 75
Score: 90
Score: 85
```

34. Processing List Data

Example:

```
scores = [80, 75, 90, 85]

total = 0

for score in scores:
    total += score

print("Total:", total)
```

Output:

```
Total: 330
```

35. Finding Values Using Loops

Example:

```
students = ["Dara", "Sokha", "Vanna"]

for student in students:
    if student == "Sokha":
        print("Student found")
        break
```

36. Nested Loops

A nested loop is a loop inside another loop.

Example:

```
for row in range(1, 4):
    for column in range(1, 4):
        print(row, column)
```

The inner loop runs completely for every iteration of the outer loop.

37. Nested Loop Execution

Example:

```
for i in range(1, 3):
    for j in range(1, 4):
        print("i =", i, "j =", j)
```

Output:

```

i = 1 j = 1
i = 1 j = 2
i = 1 j = 3
i = 2 j = 1
i = 2 j = 2
i = 2 j = 3

```

38. Nested Loop Pattern

Example:

```

for row in range(3):
    for column in range(4):
        print("*", end=" ")

    print()

```

Output:

```

* * * *
* * * *
* * * *

```

39. Multiplication Table Using Nested Loops

Example:

```

for i in range(1, 4):
    for j in range(1, 6):
        print(i, "x", j, "=", i * j)

    print()

```

Nested loops are useful for:

- Tables
- Patterns
- Coordinates
- Matrix-like data
- Multi-level records
- Repetitive combinations

40. while Loop vs for Loop

Feature	while Loop	for Loop
Main use	Repeat while condition is true	Iterate over sequence/range
Counter initialization	Often manual	Often handled by iterable/range
Counter update	Often manual	Automatic during iteration
Risk of infinite loop	Higher	Lower in ordinary finite iteration
Suitable for known count	Yes	Often preferable
Suitable for condition-based repetition	Very suitable	Sometimes
Suitable for collections	Possible	Very suitable

41. Choosing the Appropriate Loop

Use a while loop when:

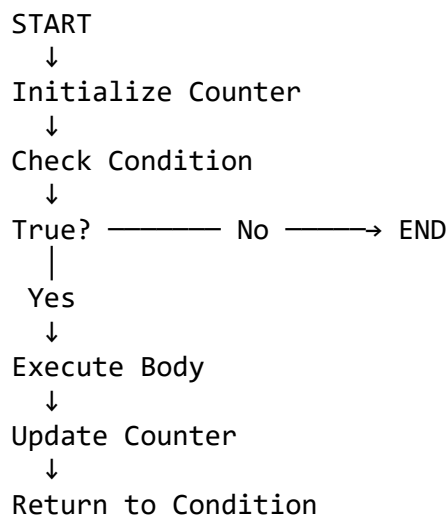
- Repetition depends mainly on a condition.
- The number of repetitions may not be known beforehand.
- Manual counter control is required.

Use a for loop when:

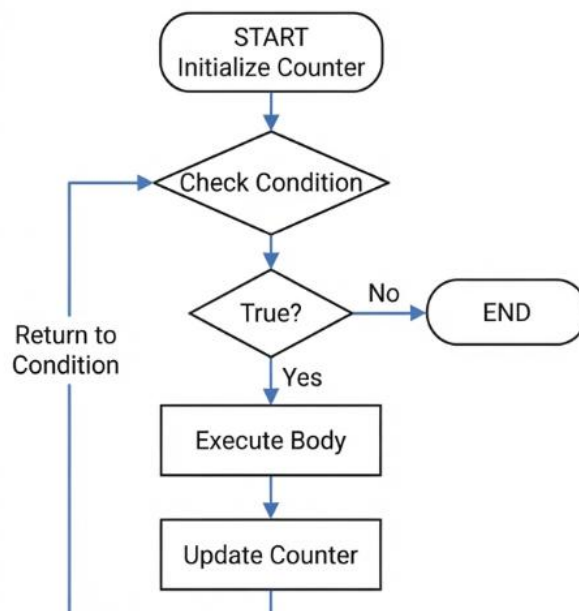
- Processing elements in a list or collection.
- Repeating a known number of times.
- Using range().
- Iterating through strings or other iterable data.

42. Loop Flow Process

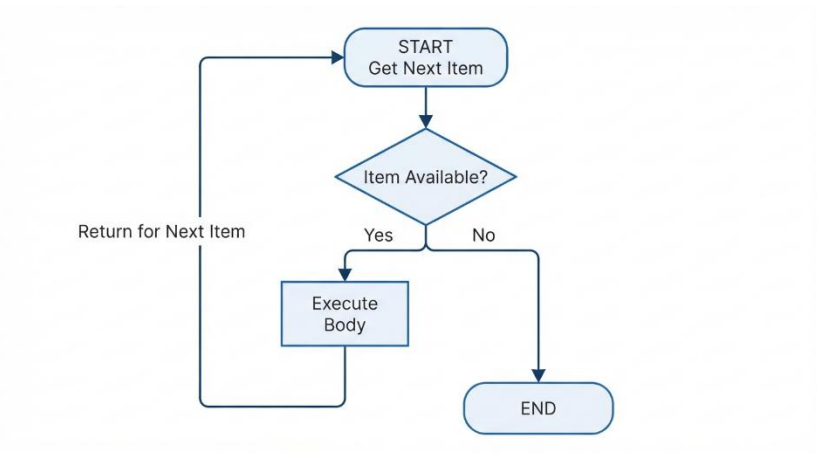
while Loop



While Loop



for Loop



43. Practical Example — Display 1 to 5 Using while

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

44. Practical Example — Display 1 to 5 Using for

```
for number in range(1, 6):
    print(number)
```

Both programs display:

```
1
2
3
4
5
```

45. Practical Example — Skip Even Numbers

```
for number in range(1, 11):
    if number % 2 == 0:
        continue

    print(number)
```

Output:

```
1
3
5
7
9
```

46. Practical Example — Stop at a Value

```
for number in range(1, 11):
    if number == 6:
        break

    print(number)
```

Output:

```
1
2
3
4
5
```

47. Practical Example — Process Student Scores

```
scores = [75, 82, 45, 90, 68]
```

```
for score in scores:
    if score >= 50:
        print(score, "Pass")
    else:
        print(score, "Fail")
```

48. Practical Example — Count Passing Scores

```
scores = [75, 82, 45, 90, 68]
```

```
pass_count = 0
```

```
for score in scores:
    if score >= 50:
        pass_count += 1

print("Pass Count:", pass_count)
```

Output:

```
Pass Count: 4
```

49. Debugging Loop Programs

When a loop does not work correctly:

1. Check variable initialization.
2. Check the loop condition.
3. Check counter updates.
4. Check indentation.
5. Check whether the loop eventually stops.
6. Check break conditions.
7. Check continue placement.
8. Check range() start value.
9. Check range() stop value.
10. Check range() step value.
11. Check nested-loop indentation.
12. Test using small values.
13. Compare actual and expected output.
14. Correct the code.
15. Execute the program again.

SELF-CHECK No. 10.1

Part A — Multiple Choice

1. What is the main purpose of a loop?

- A. Delete files
- B. Repeat program instructions
- C. Install Python
- D. Create hardware

2. Which Python keyword starts a condition-controlled loop?

- A. for
- B. while
- C. loop
- D. repeat

3. Which statement correctly initializes a counter?

- A. count == 1
- B. count = 1
- C. count : 1
- D. count -> 1

4. Which statement increments count by one?

- A. count = 1
- B. count += 1
- C. count == 1
- D. count < 1

5. Which keyword immediately stops a loop?

- A. continue
- B. stop
- C. break
- D. end

6. Which keyword skips the remainder of the current iteration?

- A. continue
- B. break
- C. pass
- D. stop

7. Which block can execute after a while loop finishes normally?

- A. elif
- B. else
- C. if
- D. case

8. Which keyword begins a sequence-based loop?

- A. for
- B. repeat
- C. do
- D. loop

9. What does range(5) generate?

- A. 1 to 5
- B. 0 to 4
- C. 0 to 5
- D. 1 to 4

10. Which range produces 1, 2, 3, 4, 5?

- A. range(5)
- B. range(1, 5)
- C. range(1, 6)
- D. range(0, 6)

11. Which range produces 2, 4, 6, 8, 10?

- A. range(2, 10)
- B. range(2, 11, 2)
- C. range(1, 10, 2)
- D. range(2, 12, 3)

12. What is a nested loop?

- A. A loop with no body
- B. A loop inside another loop
- C. A loop with an error
- D. A loop without a condition

13. What is a common cause of an infinite while loop?

- A. Correct counter update
- B. Missing counter update
- C. Using print()
- D. Using range()

14. A for loop is especially suitable for:

- A. Processing list elements
- B. Installing software
- C. Creating variables only
- D. Deleting Python

15. Which statement is valid?

- A. for i range(5):
- B. for i in range(5):
- C. for i = range(5):
- D. for range i(5):

ANSWER KEY No. 10.1

Part A

No.	Answer
1	B
2	B
3	B
4	B
5	C
6	A
7	B
8	A
9	B
10	C
11	B
12	B
13	B
14	A
15	B

OPERATION SHEET No. 10.1

Title: Create and Execute Python Loop Structures

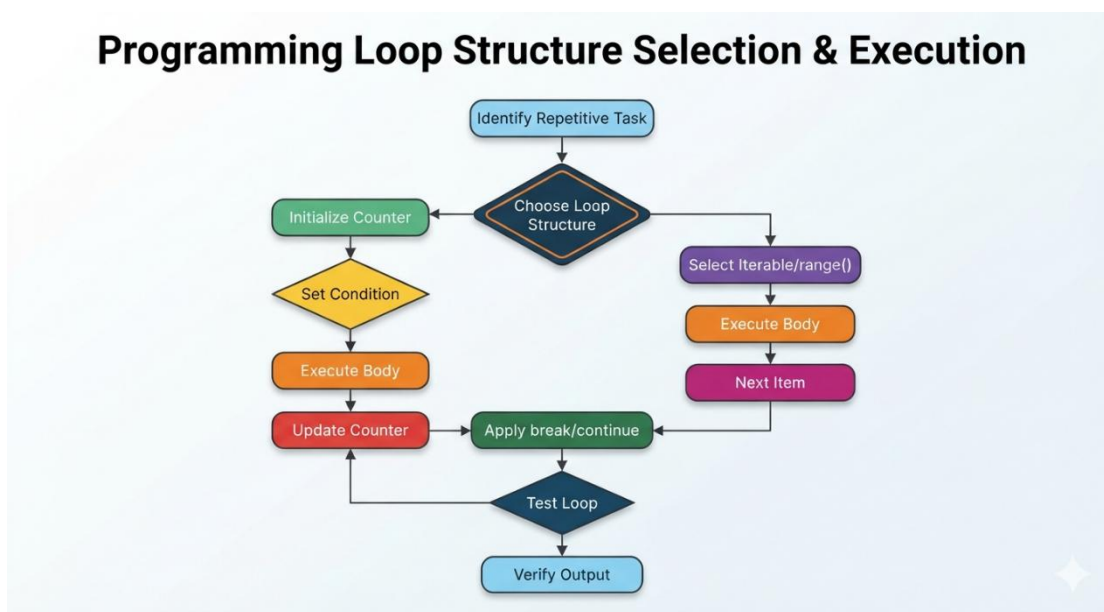
Performance Objective

Given a computer with Python installed and an approved Python development environment, the trainee must initialize loop variables, develop while and for loops, apply counters, use break, continue, and else, apply range(), create nested loops, execute the program, and verify the output.

Description

This operation demonstrates the standard procedure for developing repetitive Python programs.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/VS Code/PyCharm	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Guide	Copy	1
7	Python Loop Exercise Files	Set	1
8	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	Development environment opens successfully
2	Create new Python file	.py file is created
3	Initialize counter	Starting value is correct

No.	Procedure/Step	Key Performance
4	Declare while loop	Correct syntax is applied
5	Apply loop condition	Condition controls execution correctly
6	Print repeated statement	Output appears for each true condition
7	Increment counter	Counter changes correctly
8	Apply break	Loop stops at required condition
9	Apply continue	Required iteration is skipped
10	Apply while-else	else executes correctly
11	Create for loop	Correct syntax is used
12	Apply range()	Required number sequence is generated
13	Process collection data	Each item is handled correctly
14	Create nested loop	Inner and outer loops execute correctly
15	Save and execute program	Program runs successfully
16	Verify output	Results match requirements

Operation Procedure

Step 1 — Create a while Loop

```
count = 1
```

```
while count <= 5:
    print("Count:", count)
    count += 1
```

Step 2 — Apply break

```
count = 1
```

```
while count <= 10:
    print(count)

    if count == 5:
        break
```

```
count += 1
```

Step 3 — Apply continue

```
count = 0
```

```
while count < 5:
    count += 1

    if count == 3:
        continue
```

```
print(count)
```

Step 4 — Apply while-else

```
count = 1
```

```
while count <= 3:
    print(count)
    count += 1
else:
    print("Loop completed")
```

Step 5 — Apply for Loop

```
for number in range(1, 6):
    print(number)
```

Step 6 — Process a List

```
students = ["Dara", "Sokha", "Vanna"]
```

```
for student in students:
    print(student)
```

Step 7 — Create Nested Loop

```
for row in range(1, 4):
    for column in range(1, 4):
        print(row, column)
```

Save as:

```
loop_operations.py
```

Execute and verify all outputs.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 10.0

Title: Develop a Counter-Controlled while Loop

Performance Task

Initialize a counter variable and use a while loop to repeatedly display values until the condition becomes false.

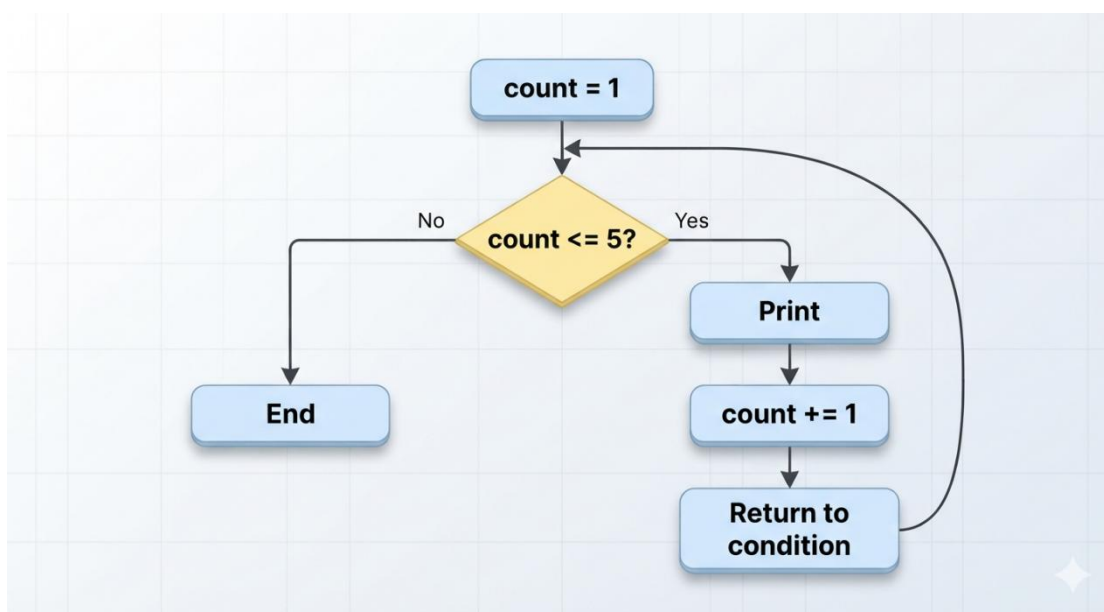
Performance Objective

Given Python IDLE/IDE, the trainee must initialize a counter, declare a while loop, display output while the condition is true, increment the counter, and stop when the condition becomes false.

Description

The trainee practices the core components of a counter-controlled while loop.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Initialize count	Starting value is assigned
2	Declare while	Correct syntax and colon are used

No.	Procedure/Step	Key Performance
3	Apply condition	Loop runs while condition is true
4	Print count	Correct value is displayed
5	Increment count	Counter increases by one
6	Stop loop	Loop ends when condition becomes false
7	Verify output	Expected sequence is produced

Practical Exercise

```
count = 1
```

```
while count <= 5:
    print("Count:", count)
    count += 1
```

Expected:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Complete:

Iteration	count before increment	Condition
1	_____	_____
2	_____	_____
3	_____	_____
4	_____	_____
5	_____	_____
After loop	_____	False

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

TASK SHEET No. 10.1

Title: Apply break, continue, and while-else

Performance Task

Apply break, continue, and else in Python while loop structures.

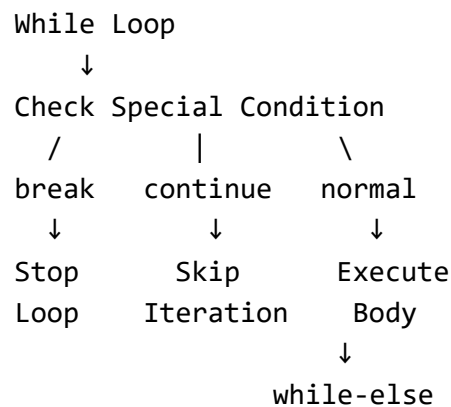
Performance Objective

Given specified loop requirements, the trainee must correctly terminate a loop, skip an iteration, and execute an else block after normal loop completion.

Description

This task develops control over while loop execution.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No	Procedure/Step	Key Performance
1	Create loop counter	Correct initial value is assigned
2	Apply break condition	Loop terminates correctly
3	Apply continue	Required iteration is skipped
4	Maintain counter update	Infinite loop is avoided
5	Apply while-else	else executes after normal completion
6	Test break with else	else is skipped after break
7	Verify outputs	Results match requirements

Exercise A — break

```
count = 1
```

```
while count <= 10:
    if count == 6:
```

```
break
```

```
print(count)  
count += 1
```

Expected:

```
1  
2  
3  
4  
5
```

Exercise B — continue

```
count = 0
```

```
while count < 5:  
    count += 1
```

```
    if count == 3:  
        continue
```

```
print(count)
```

Expected:

```
1  
2  
4  
5
```

Exercise C — while-else

```
count = 1
```

```
while count <= 3:  
    print(count)  
    count += 1  
else:  
    print("Completed")
```

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Output verification

TASK SHEET No. 10.2

Title: Develop Python for Loops Using range()

Performance Task

Develop for loops using different forms of the range() function.

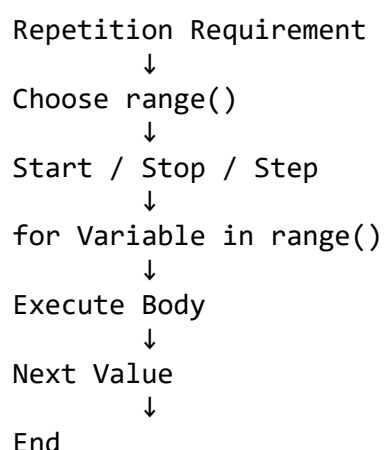
Performance Objective

Given repetition requirements, the trainee must correctly apply range(stop), range(start, stop), and range(start, stop, step).

Description

The trainee practices controlling repetitive for loops with generated integer sequences.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create for loop	Correct syntax is applied
2	Use range(5)	0 through 4 are produced
3	Use range(1, 6)	1 through 5 are produced
4	Apply step value	Required interval is generated
5	Apply negative step	Backward sequence is produced
6	Execute program	Program runs successfully

No.	Procedure/Step	Key Performance
7	Verify sequence	Actual output matches requirement

Practical Exercise

```

print("Range 1")
for number in range(5):
    print(number)

print("Range 2")
for number in range(1, 6):
    print(number)

print("Even Numbers")
for number in range(2, 11, 2):
    print(number)

print("Countdown")
for number in range(5, 0, -1):
    print(number)

```

Complete:

Expression	Generated Values
range(5)	_____
range(1, 6)	_____
range(2, 11, 2)	_____
range(5, 0, -1)	_____

Assessment Method

- Direct observation
- Practical exercise
- Worksheet review
- Output verification

TASK SHEET No. 10.3

Title: Process List Data Using for Loops

Performance Task

Use a for loop to process student scores stored in a list.

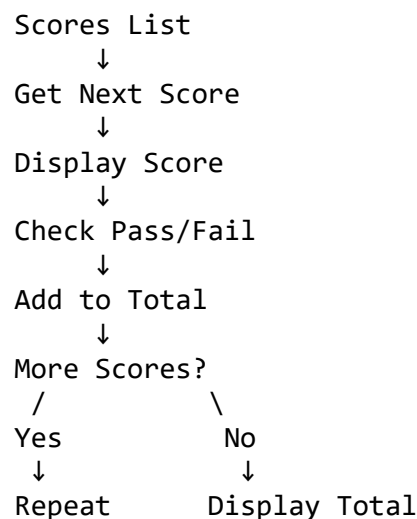
Performance Objective

Given a list of scores, the trainee must iterate through the list, display each value, classify pass/fail results, and calculate a total.

Description

This task applies loops to collection data processing.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create score list	Required values are stored
2	Create for loop	Each score is processed
3	Display values	All scores are shown
4	Apply conditional	Pass/fail is correctly determined
5	Initialize total	Total begins correctly

No.	Procedure/Step	Key Performance
6	Accumulate scores	All values are added
7	Display total	Correct sum is shown
8	Verify results	Output meets requirements

Practical Exercise

```
scores = [75, 82, 45, 90, 68]
```

```
total = 0
```

```
for score in scores:
    print("Score:", score)

    if score >= 50:
        print("Result: Pass")
    else:
        print("Result: Fail")

    total += score

print("Total:", total)
```

Expected total:

360

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

TASK SHEET No. 10.4

Title: Apply break and continue in for Loops

Performance Task

Use break and continue to control for loop iterations.

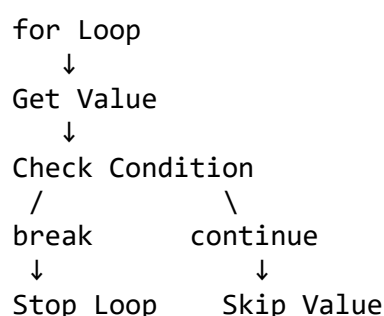
Performance Objective

Given numeric sequences, the trainee must stop a for loop at a specified value and skip specified iterations.

Description

The trainee applies loop-control statements to for loops.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Practical Exercise A — break

```
for number in range(1, 11):  
    if number == 6:  
        break  
  
    print(number)
```

Practical Exercise B — continue

```
for number in range(1, 11):  
    if number % 2 == 0:  
        continue  
  
    print(number)
```

Expected output of Exercise B:

1
3
5
7
9

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create for loop	Correct syntax is used
2	Apply condition	Required value is identified
3	Apply break	Loop stops correctly
4	Apply continue	Required iterations are skipped
5	Execute both programs	Programs run successfully
6	Verify output	Results match requirements

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 10.5

Title: Develop Nested Loop Structures

Performance Task

Develop nested loops to perform repetitive multi-level tasks.

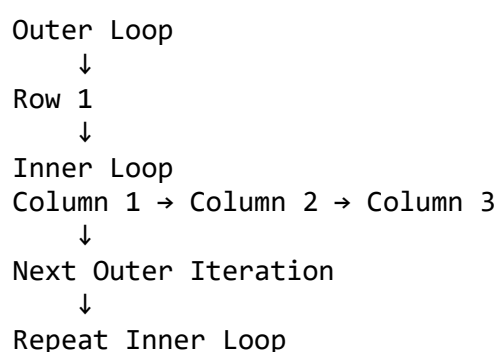
Performance Objective

Given row and column requirements, the trainee must create an outer loop and inner loop and verify the number and arrangement of repeated operations.

Description

The trainee uses nested loops to create a simple pattern and multiplication table.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create outer loop	Required row iterations occur
2	Create inner loop	Required column iterations occur
3	Apply correct indentation	Inner loop is nested correctly
4	Display pattern	Required pattern is produced
5	Create multiplication task	Correct products are displayed
6	Test loop counts	Number of iterations is correct
7	Verify output	Results meet requirements

Exercise A — Pattern

```
for row in range(3):
    for column in range(4):
        print("*", end=" ")

    print()
```

Expected:

```
* * * *
* * * *
* * * *
```

Exercise B — Multiplication

```
for i in range(1, 4):
    for j in range(1, 6):
        print(i, "x", j, "=", i * j)

    print()
```

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

JOB SHEET No. 10.1

Title: Develop a Student Score Processing Program Using Loop Structures

Performance Task

Develop a complete Python program using while, for, range(), break, continue, else, collection processing, and nested loops.

Performance Objective

Given a computer, Python development environment, student-score data, and program requirements, the trainee must initialize loop variables, develop while and for loop structures, control execution using counters and conditions, apply loop-control statements, process list data, develop a nested loop, test the program, debug errors, and verify the final output.

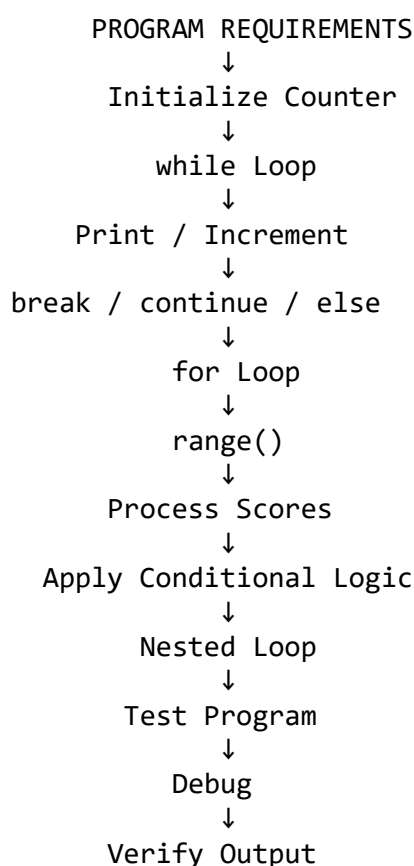
Description

Develop:

`student_loop_processor.py`

The program will demonstrate all major loop competencies required by LO10.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1

No.	Description	Unit	Qty
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Python Loop Exercise Files	Set	1
6	Python Programming Reference Guide	Copy	1
7	Programming Worksheet	Copy	1
8	USB Flash Drive	Unit	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze repetitive requirements	Appropriate loop structures are identified
2	Open Python IDE	Development environment opens successfully
3	Create program file	.py file is created
4	Initialize counter	Starting value is correct
5	Create while loop	Correct condition is applied
6	Display repeated statements	Output appears while condition is true
7	Increment counter	Counter changes correctly
8	Apply break	Loop stops at required point
9	Apply continue	Specified iteration is skipped
10	Apply while-else	Normal completion message is displayed
11	Create for loop	Correct syntax is applied
12	Apply range()	Required sequence is generated
13	Process list data	All list items are processed
14	Apply conditional inside loop	Pass/fail logic is correct
15	Accumulate results	Totals/counts are correct
16	Create nested loop	Repetitive multi-level task is completed
17	Save program	.py extension is used
18	Execute program	Program runs successfully
19	Test loop termination	All loops stop correctly
20	Check for infinite loops	Counter logic is valid
21	Debug identified errors	Corrections are applied
22	Verify final output	Results match requirements

Program Requirements

The program must demonstrate:

1. Counter initialization.
2. while loop.

3. Counter increment.
4. Repeated output.
5. break.
6. continue.
7. while-else.
8. for loop.
9. range().
10. Looping through a list.
11. Conditional statement within a loop.
12. Running total or count.
13. Nested loops.
14. Program testing.

Part 1 — Basic while Loop

```
count = 1

while count <= 5:
    print("Count:", count)
    count += 1
else:
    print("Basic while loop completed")
```

Part 2 — break

```
number = 1

while number <= 10:
    if number == 6:
        print("Stopping at 6")
        break

    print(number)
    number += 1
```

Part 3 — continue

```
number = 0

while number < 5:
    number += 1

    if number == 3:
        continue

    print("Continue Example:", number)
```

Part 4 — for Loop with range()

```
for number in range(1, 6):
    print("For Loop:", number)
```

Part 5 — Process Student Scores

Create:

```
scores = [75, 82, 45, 90, 68]
```

For every score:

- Display the score.
- Print Pass when the score is at least 50.
- Print Fail otherwise.
- Calculate total scores.
- Count passing scores.

Part 6 — Nested Loop

Create a 3 × 5 multiplication table.

```
for row in range(1, 4):
    for column in range(1, 6):
        print(row, "x", column, "=", row * column)

    print()
```

Example Complete Program

```
# Student Loop Processing Program
```

```
print("=====")
print("      LOOP DEMONSTRATION")
print("=====")
```

```
# 1. Basic while loop
```

```
print("\nWHILE LOOP")
```

```
count = 1
```

```
while count <= 5:
    print("Count:", count)
    count += 1
else:
    print("While loop completed")
```

```
# 2. break
```

```
print("\nBREAK EXAMPLE")
```

```
number = 1
```

```
while number <= 10:
    if number == 6:
        print("Break at:", number)
        break
```

```
print(number)
number += 1
```

```

# 3. continue
print("\nCONTINUE EXAMPLE")

number = 0

while number < 5:
    number += 1

    if number == 3:
        continue

    print(number)

# 4. for Loop and range()
print("\nFOR LOOP WITH RANGE")

for number in range(1, 6):
    print(number)

# 5. Student scores
print("\nSTUDENT SCORE PROCESSING")

scores = [75, 82, 45, 90, 68]

total = 0
pass_count = 0

for score in scores:
    print("Score:", score)

    if score >= 50:
        print("Result: Pass")
        pass_count += 1
    else:
        print("Result: Fail")

    total += score

print("Total Score:", total)
print("Pass Count:", pass_count)

# 6. Nested Loop
print("\nMULTIPLICATION TABLE")

for row in range(1, 4):
    for column in range(1, 6):
        print(row, "x", column, "=", row * column)

    print()

```

Expected Main Results

while Loop

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
While loop completed
```

break

```
1
2
3
4
5
Break at: 6
```

continue

```
1
2
4
5
```

for Loop

```
1
2
3
4
5
```

Student Score Processing

Given:

```
75, 82, 45, 90, 68
```

Expected:

```
Total Score: 360
Pass Count: 4
```

Testing Requirements

Test Item	Expected Result	Actual Result	Pass/Fail
while loop starts at	1	_____	_____
while loop final printed count	5	_____	_____
while else executes	Yes	_____	_____
break occurs at	6	_____	_____
continue skips	3	_____	_____
range(1, 6) final value	5	_____	_____
Score total	360	_____	_____

Test Item	Expected Result	Actual Result	Pass/Fail
Pass count	4	_____	_____
Nested outer iterations	3	_____	_____
Nested inner iterations per row	5	_____	_____

Debugging Requirements

Error 1 — Infinite Loop

Incorrect:

```
count = 1

while count <= 5:
    print(count)
```

Correction:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Error 2 — Incorrect continue Placement

Incorrect:

```
count = 1

while count <= 5:
    if count == 3:
        continue

    print(count)
    count += 1
```

The trainee must explain why this can become an infinite loop and correct the counter logic.

Error 3 — Incorrect range()

Requirement:

Display 1 to 5.

Incorrect:

```
for number in range(1, 5):
    print(number)
```

Output only reaches 4.

Correct:

```
for number in range(1, 6):
    print(number)
```

Error 4 — Incorrect Nested Indentation

Incorrect:

```
for row in range(3):  
    for column in range(4):  
        print("*")
```

Correct:

```
for row in range(3):  
    for column in range(4):  
        print("*")
```

Assessment Method

- Oral questioning
- Written/theoretical assessment
- Practical workshop assessment
- Direct observation
- Source-code inspection
- Program execution
- Output verification
- Debugging demonstration

WORKSHEET No. 10.1

Title: Python Loop Structure Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — Identify Loop Components

Given:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Identify:

Initialization: _____

Condition: _____

Loop Statement: _____

Counter Update: _____

Part B — Predict while Loop Output

Given:

```
count = 1

while count <= 3:
    print(count)
    count += 1
```

Output:

Part C — Increment and Decrement

Write a statement to increment count by one:

Write a statement to decrement count by one:

Part D — break

Complete:

```
for number in range(1, 10):
    if number == 5:
        _____

    print(number)
```

Part E — continue

Complete:

```
for number in range(1, 6):  
    if number == 3:  
        _____  
  
    print(number)
```

Part F — while-else

Complete:

```
count = 1  
  
while count <= 3:  
    print(count)  
    count += 1  
    _____:  
    print("Completed")
```

Part G — range()

Complete:

Expression	Output Values
range(5)	_____
range(1, 6)	_____
range(2, 11, 2)	_____
range(10, 0, -2)	_____

Part H — for Loop

Complete:

```
students = ["Dara", "Sokha", "Vanna"]  
  
_____ student _____ students:  
    print(student)
```

Answers:

Part I — Process Collection Data

Given:

```
scores = [80, 40, 70, 45, 90]
```

Write a for loop to display each score:

Part J — Nested Loop

Complete:

```

for row in range(3):
    _____ column in range(4):
        print("*", end=" ")
    print()

```

Answer: _____

Part K — Infinite Loop

Why does this program not stop?

```

count = 1

while count <= 5:
    print(count)

```

Answer: _____

Correct it:

Part L — Choose the Loop

Write **while** or **for**.

Requirement	Recommended Loop
Repeat while password is incorrect	_____
Display numbers 1–10	_____
Process every item in a list	_____
Repeat until a condition becomes false	_____
Process every character in a string	_____

PERFORMANCE CHECKLIST No. 10.1

Title: Develop Program Using Loop Structure

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Explained the purpose of Python loops	☐	☐
2	Initialized a variable with an appropriate value	☐	☐
3	Created a counter variable correctly	☐	☐
4	Declared a while loop using correct syntax	☐	☐
5	Applied an appropriate while condition	☐	☐
6	Printed statements while the condition was true	☐	☐
7	Incremented the counter correctly	☐	☐
8	Decrementing a counter correctly where required	☐	☐
9	Ensured the condition eventually became false	☐	☐
10	Avoided an unintended infinite loop	☐	☐
11	Applied break correctly	☐	☐
12	Applied an appropriate condition for break	☐	☐
13	Applied continue correctly	☐	☐
14	Maintained correct counter logic when using continue	☐	☐
15	Applied while-else correctly	☐	☐
16	Recognized the effect of break on a loop else block	☐	☐
17	Applied comparison operators in loop conditions	☐	☐
18	Applied logical operators in loop conditions where required	☐	☐
19	Declared a for loop using correct Python syntax	☐	☐
20	Iterated through a list using a for loop	☐	☐
21	Iterated through a sequence correctly	☐	☐
22	Applied range(stop) correctly	☐	☐
23	Applied range(start, stop) correctly	☐	☐
24	Applied range(start, stop, step) correctly	☐	☐
25	Applied a negative step in range() where required	☐	☐
26	Combined a for loop with a conditional statement	☐	☐
27	Applied break in a for loop	☐	☐
28	Applied continue in a for loop	☐	☐
29	Applied loops to process list/collection data	☐	☐
30	Calculated a running total correctly	☐	☐
31	Used a loop counter to count matching records	☐	☐
32	Created a nested loop correctly	☐	☐
33	Applied correct indentation to nested loops	☐	☐
34	Completed repetitive tasks using nested loops	☐	☐

No.	Criteria	Yes	No
35	Selected an appropriate loop for the program requirement	☐	☐
36	Saved the program with .py extension	☐	☐
37	Executed the loop program successfully	☐	☐
38	Tested loop start values	☐	☐
39	Tested loop stopping conditions	☐	☐
40	Tested break conditions	☐	☐
41	Tested continue conditions	☐	☐
42	Tested range() boundaries	☐	☐
43	Tested nested loops	☐	☐
44	Identified counter-related errors	☐	☐
45	Identified infinite-loop errors	☐	☐
46	Identified indentation errors	☐	☐
47	Applied appropriate debugging techniques	☐	☐
48	Corrected identified loop errors	☐	☐
49	Verified final program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____